

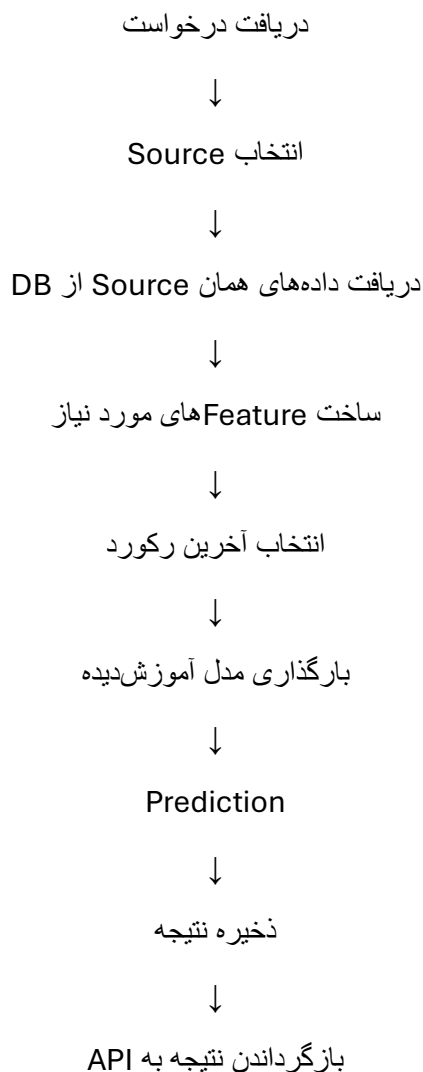
به نام خداوند بخشنده مهربان

1. هدف سرویس

سرویس پیش‌بینی مسئول اجرای فرآیند پیش‌بینی قیمت نقره در زمان دریافت درخواست از API است. در این بخش، مدل آموزش داده نمی‌شود. مدل‌های آموزش‌دیده قبلی از فایل‌های ذخیره‌شده بارگذاری شده و با استفاده از آخرین داده موجود در دیتابیس، قیمت بعدی پیش‌بینی می‌شود. از آنجا که برای هر منبع مدل مستقل وجود دارد، فرآیند پیش‌بینی نیز برای هر منبع به صورت جداگانه انجام می‌شود.

2. تفکیک Training و Prediction

یکی از تصمیمات اصلی در طراحی این بخش، جدا کردن فرآیند آموزش مدل از فرآیند پیش‌بینی است. در فرآیند Training، داده‌های تاریخی مورد استفاده قرار می‌گیرند و مدل‌ها آموزش داده شده و در فایل‌های "pkl." ذخیره می‌شوند. اما در Prediction، مدل مجدداً آموزش داده نمی‌شود. فرآیند Prediction تنها شامل موارد زیر است:



این تفکیک باعث می‌شود درخواست‌های API نیازی به اجرای فرآیند زمان‌بر Training نداشته باشند.

3. انتخاب منبع

در سرویس، منابع قابل استفاده از قبل مشخص شده‌اند:

```
SOURCES = ("tgju", "silfam", "noghrese")
```

هنگام دریافت درخواست، Source بررسی می‌شود تا فقط منابع تعریف‌شده امکان Prediction داشته باشند.

```
if source not in SOURCES:
```

```
    raise ValueError(f"Unsupported source: {source}")
```

این کنترل باعث می‌شود سرویس برای یک منبع ناشناخته یا نامعتبر اقدام به جست‌وجوی مدل و داده نکند. همچنین مقدار Source از طریق "SourceEnum" در Schema درخواست کنترل می‌شود تا در سطح API نیز ورودی‌های معتبر محدود شوند.

4. دریافت داده‌های آخرین وضعیت

برای Prediction نمی‌توان فقط آخرین قیمت را از دیتابیس دریافت کرد. دلیل این است که Feature ها به چند رکورد قبلی نیاز دارند. بنابراین سرویس ابتدا تمام داده‌های همان Source را از طریق "load_price_data()" دریافت می‌کند.

```
data = load_price_data(db, source)
```

سپس داده‌ها بر اساس زمان مرتب شده و Feature های موردنیاز روی آن‌ها ساخته می‌شوند. در نتیجه، آخرین رکورد پس از محاسبه Feature ها نمایانگر آخرین وضعیت قابل استفاده برای Prediction است.

5. ساخت Feature های Prediction

تابع "prepare_prediction_features()" وظیفه دارد داده‌های خام Source را دریافت کرده و Feature های موردنیاز مدل را ایجاد کند. Feature ها با همان توابعی ساخته می‌شوند که در فرآیند Training استفاده شده‌اند. این موضوع اهمیت زیادی دارد، زیرا مدل در زمان Training با ساختار مشخصی از Feature ها آموزش دیده است و در زمان Prediction نیز باید دقیقاً همان Feature ها را دریافت کند.

پس از ساخت Feature ها، آخرین رکورد انتخاب می‌شود:

```
return data.iloc[:1]
```

در اینجا "next_price" موردنیاز نیست، زیرا در زمان Prediction قیمت آینده هنوز مشخص نیست. این مقدار فقط در Training به عنوان Target استفاده می‌شود.

6. استفاده از مدل ذخیره شده

برای هر Source و هر الگوریتم، یک فایل مدل جداگانه ذخیره می شود. ساختار فایل ها به صورت زیر است:

/ml_models

tgju_LinearRegression.pkl — |

tgju_RandomForestRegressor.pkl — |

silfam_LinearRegression.pkl — |

silfam_RandomForestRegressor.pkl — |

noghresea_LinearRegression.pkl — |

noghresea_RandomForestRegressor.pkl — L

نام فایل بر اساس Source و نام مدل ساخته می شود:

```
model_path = MODEL_DIR / f"{source}_{model_name}.pkl"
```

قبل از بارگذاری، وجود فایل بررسی می شود:

```
if not model_path.exists():
```

```
    raise FileNotFoundError(
```

```
        f"Model file not found: {model_path}")
```

سپس مدل با "joblib" بارگذاری می شود:

```
model = joblib.load(model_path)
```

در نتیجه سرویس از همان مدلی استفاده می کند که قبلاً توسط فرآیند Training آموزش داده و ذخیره شده است.

7. آماده سازی ورودی مدل

پس از ساخت Feature ها، فقط ستون هایی که مدل انتظار دارد انتخاب می شوند:

```
features = latest[FEATURES]
```

استفاده از "FEATURES" باعث می شود ترتیب و نام Feature های ورودی با داده ای که مدل در زمان Training دریافت کرده یکسان باشد. این مسئله از ورود ستون های اضافی یا تغییر ترتیب Feature ها جلوگیری می کند.

8. انجام Prediction

پس از آماده شدن ورودی، مدل Prediction را انجام می دهد:

```
predicted_price = int(round(model.predict(features)[0]))
```

مدل یک مقدار عددی پیش‌بینی می‌کند. سپس مقدار پیش‌بینی‌شده گرد شده و به "int" تبدیل می‌شود تا قیمت نهایی به صورت عدد صحیح ذخیره و ارائه شود.

9. ذخیره نتیجه Prediction

نتیجه Prediction فقط به کاربر بازگردانده نمی‌شود، بلکه در جدول "predictions" نیز ذخیره می‌شود. اطلاعات ذخیره‌شده شامل موارد زیر است:

- قیمت پیش‌بینی‌شده

- نام مدل

- زمان Prediction

این کار امکان ثبت تاریخچه Predictionها و بررسی عملکرد مدل در آینده را فراهم می‌کند.

10. مدیریت خطا

سرویس تمام فرآیند Prediction را در یک "try/except" قرار داده است. در صورت بروز خطا، ابتدا Transaction دیتابیس Rollback می‌شود:

```
db.rollback()
```

سپس خطا همراه با اطلاعات Source و مدل در Logger ثبت می‌شود:

```
logger.exception(
```

```
    "Prediction failed for %s using %s",
```

```
    source,
```

```
    model_name,
```

```
)
```

در نهایت خطا دوباره با "raise" ارسال می‌شود تا لایه بالاتر، یعنی API، بتواند پاسخ HTTP مناسب را ایجاد کند. این طراحی باعث می‌شود خطاها در سرویس مخفی نشوند و در عین حال Transaction دیتابیس نیز در وضعیت نامعتبر باقی نماند.

11. تفکیک مدل‌های Linear Regression و Random Forest

برای هر Source دو مدل مستقل وجود دارد:

Source

Linear Regression — |

Random Forest — L

بنابراین Prediction Service برای هر درخواست می‌تواند Prediction هر دو مدل را اجرا کند. برای جلوگیری از تکرار منطق اصلی، دو تابع کوچک برای فراخوانی مدل‌ها وجود دارد:

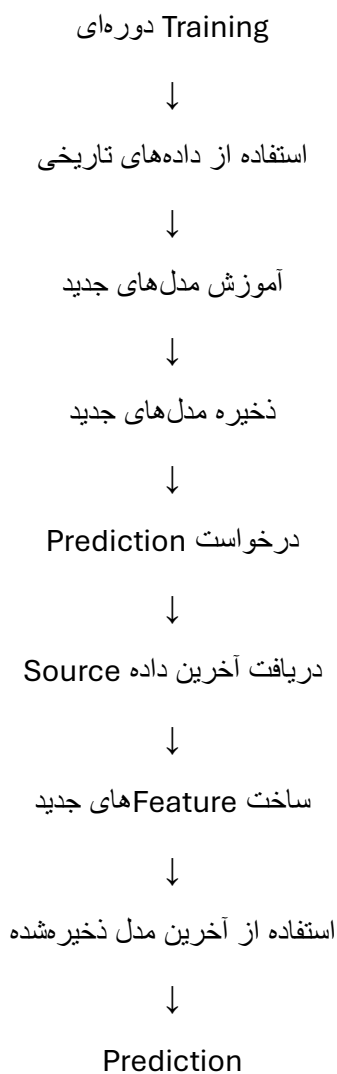
`predict_linear_regression(db, source)`

`predict_random_forest(db, source)`

هر دو تابع در نهایت از تابع عمومی "`predict()`" استفاده می‌کنند. به این ترتیب منطق اصلی Prediction تنها در یک محل پیاده‌سازی شده است.

12. چرخه به‌روزرسانی مدل و Prediction

مدل‌ها قرار نیست با هر درخواست API دوباره آموزش داده شوند. برنامه‌ریزی در نظر گرفته‌شده برای سیستم به این صورت است:



بنابراین دو مفهوم از یکدیگر مستقل هستند:

Training دوره‌ای، مدل را به‌روزرسانی می‌کند. Prediction لحظه‌ای، آخرین داده موجود را به مدل فعلی می‌دهد. در آینده می‌توان Training را به صورت یک Job هفتگی در Scheduler اجرا کرد، بدون اینکه ساختار سرویس Prediction تغییر کند.

13. تصمیمات طراحی و دلایل آن‌ها

استفاده از مدل مستقل برای هر Source:

به دلیل تفاوت رفتار و مقیاس قیمت‌ها در منابع مختلف، داده‌های منابع با یکدیگر ترکیب نمی‌شوند و برای هر Source مدل‌های جداگانه آموزش داده می‌شوند. در نتیجه مدل TGJU فقط با داده‌های TGJU، مدل Silfam با داده‌های Silfam و مدل NogrehSea با داده‌های NogrehSea کار می‌کند.

استفاده از آخرین داده در زمان Prediction:

هدف Prediction، پیش‌بینی قیمت بعدی بر اساس جدیدترین وضعیت موجود است. بنابراین در زمان درخواست API، داده‌های موجود در دیتابیس مجدداً خوانده می‌شوند و Feature‌های آخرین وضعیت محاسبه می‌شوند. این باعث می‌شود حتی اگر مدل آخرین بار چند روز قبل آموزش دیده باشد، Prediction بر اساس جدیدترین داده‌های ثبت‌شده انجام شود.

عدم استفاده از "next_price" در Prediction:

"next_price" مقدار آینده است و در زمان Prediction هنوز وجود ندارد. بنابراین این ستون فقط در Training به عنوان Target استفاده می‌شود و وارد ورودی مدل نمی‌شود.

استفاده مجدد از توابع Feature Engineering:

Feature‌های Training و Prediction باید دقیقاً یکسان باشند. به همین دلیل به جای نوشتن دوباره منطق محاسبه Feature‌ها در Service، همان توابع موجود در ماژول Feature Engineering استفاده می‌شوند. این کار از ایجاد اختلاف بین داده Training و داده Prediction جلوگیری می‌کند.

14. نتیجه

Prediction Service به عنوان واسط بین دیتابیس، Feature Engineering و مدل‌های ذخیره‌شده عمل می‌کند. این سرویس مسئول آموزش مدل نیست؛ بلکه در هر درخواست:

1. Source معتبر را دریافت می‌کند.
2. داده‌های همان Source را از دیتابیس می‌گیرد.
3. Feature‌های لازم را ایجاد می‌کند.
4. آخرین وضعیت موجود را استخراج می‌کند.
5. مدل مناسب Source و الگوریتم را بارگذاری می‌کند.
6. قیمت بعدی را پیش‌بینی می‌کند.
7. نتیجه را در دیتابیس ذخیره می‌کند.
8. نتیجه را برای لایه API ارسال می‌کند.

به این ترتیب، Training دوره‌ای و Prediction لحظه‌ای از یکدیگر مستقل بوده و هر کدام مسئولیت مشخصی در معماری سیستم دارند.